5-4 · DataFrame과 SparkSQL 가공 · 실습 1.5H

DataFrame API로 데이터 가공

조인·파생·필터·집계를 체인으로 — 중간마다 확인

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① orders·products 를 만들고 order_date 를 date 로 바꾼다

② join → withColumn → filter → groupBy 를 체인으로 잇는다

③ ambiguous · cannot resolve 두 오류를 직접 내고 고친다

④ 윈도로 날짜별 1위를 뽑고 explain() 에서 조인 방식을 읽는다

cd ~/data-eng/week5 → bash lake-pyspark.sh 로 셸을 띄웁니다 — 이후는 >>> 안

실습 흐름

여덟 단계 — 표 두 개에서 실행 계획까지

3주차 Pandas 에서 한 일을 분산 환경에서 그대로 — 문법이 조금 다를 뿐 생각은 같습니다

1·2. 셀·타입	3. 조인	4. 일부러 실패	5·6. 필터·집계	7·8. 윈도·계획
5행·3행	5행 → 5행	ambiguous	big 2행	row_number
to_date	amount 파생	cannot resolve	groupBy	explain()

1·2단계

셸을 띄우고 표 두 개 · 날짜 타입

1. **bash lake-pyspark.sh**
   「0 artifacts copied, 4 already retrieved」— 10~20초 뒤 >>>
2. **createDataFrame 으로 orders · products**
   orders 5행 · products 3행 — printSchema() 에 order_date: string
3. **F.to_date("order_date") 로 덮어쓰기**
   withColumn 뒤 printSchema() 에 order_date: date 면 통과

먼저 한 줄

functions

as F

> **Windows(WSL2) 분 — 0 artifacts copied 가 나왔나요, jar 를 다시 받았나요?**

3단계

조인하고 amount 를 만든다 — 5행 → 5행

1. **orders.join(products, on="product_id", how="inner")**
「조인 전: 5 행 / 조인 후: 5 행」 · product_id 칸은 하나만

2. **withColumn("amount", F.col("qty") * F.col("price"))**
225000 · 90000 · 46000 · 23000 · 930000 다섯 값(순서는 다를 수 있음)

3. **products.select("product_id").distinct().count()**
「products 행: 3」 · 「product_id 고유: 3」 — validate 대신 이 한 줄

두 숫자

3 = 3

이면 안전

4단계 · 일부러 실패시키기

조인 키를 조건식으로 주면 — ambiguous

이론의 「키를 문자열로」가 왜 규칙인지 — 오류를 한 번 직접 봅니다

1. 조건식 조인	2. bad.columns	3. 오류	4. 처방	5. 한 번 더
orders["product_id"] == products[...]	product_id 가 두 번	AMBIGUOUS_REFERENCE	on="product_id" 또는 alias	joined 에 amount → cannot resolve

4단계 · 일부러 실패시키기

두 오류 — ambiguous 와 cannot resolve

[AMBIGUOUS_REFERENCE]

Reference `product_id` is ambiguous

- 원인: 조건식 조인은 두 컬럼을 다 남긴다
- 처방 1: on="product_id" 문자열로
- 처방 2: alias("o")·alias("p") 로 출처 명시

VS

[UNRESOLVED_COLUMN...]

`amount` cannot be resolved

- 원인: 가공 전 변수 joined 에는 amount 가 없다
- .WITH_SUGGESTION — 그 시점의 컬럼 후보
- 처방: with_amount 에서 filter

5~7단계

필터 · 집계 · 윈도 — 결과 값으로 확인

1. **filter(F.col("amount") >= 100000)**
   big 은 정확히 2행 — 3 keyboard 225000 · 4 monitor 930000 · recent_big 도 같음

2. **groupBy("name").agg(...).orderBy(F.desc(...))**
   monitor 930000·3 / keyboard 315000·7 / mouse 69000·3

3. **F.row_number().over(w) → filter("rn = 1")**
   08-24 keyboard 90000 · 08-25 monitor 930000 · 08-26 mouse 46000

복합 조건

(조건1) &

(조건2)

8단계

sales.explain() — SortMergeJoin 과 Exchange 넷

1. **sales.explain() → SortMergeJoin**
   Exchange 넷 — 조인 양쪽 hashpartitioning(product_id) 둘 · name · range

2. **왜 Broadcast 가 아닌가**
   createDataFrame 표는 크기를 몰라 약 6.9×10^{18} 바이트로 봄 → 10MB 초과

3. **orders.join(F.broadcast(products), ...)**
   BroadcastHashJoin · BroadcastExchange — 조인 쪽 Exchange 가 사라짐

Iceberg 표

크기를 알아

힌트 없이

> **lake.db.orders 는 144 바이트로 계산 — 힌트 없이 Broadcast. explain 결과가 SortMergeJoin 으로 같게 나왔나요?**

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. 0 artifacts copied 뒤 >>> · order_date 가 date 다
2. 조인 전후 5 = 5 · products 키 고유성 3 = 3
3. AMBIGUOUS_REFERENCE · UNRESOLVED_COLUMN 을 직접 봤다
4. big 정확히 2행 · 상품별 합계 monitor · keyboard · mouse 순
5. 날짜별 1위 세 행 · 08-24 keyboard 90000 부터
6. SortMergeJoin · Exchange 넷 → broadcast 힌트로 BroadcastHashJoin

5-4 · 실습 1.5H 마무리

다음은 SparkSQL 실습 (1H)

같은 가공을 SQL 로 다시 쓰고, 두 방식의 explain() 을 대조한 뒤 Iceberg 테이블로 저장합니다

- ✓ **createOrReplaceTempView 로 임시 뷰**
- ✓ **SHOW VIEWS IN default 로 확인**
- ✓ **SQL 로 같은 집계 — 결과 대조**
- ✓ **lake.db.product_sales 로 저장**

가져갈 것

셀 그대로 · sales · with_amount